

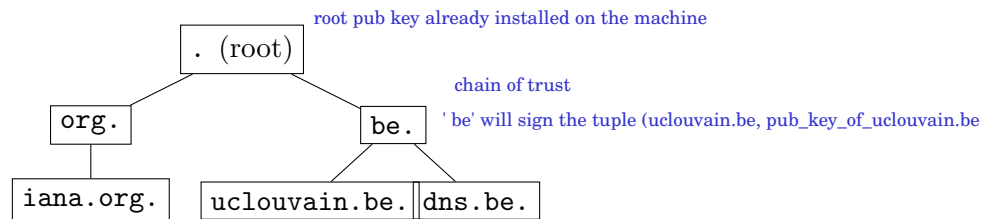
LINFO2144 - Practical session 4:

Protocols and data structures for security

The following exercises are all inspired from June 2025's exam exercises. This lab is shorter than the previous ones, if you still have time at the end you can finish the exercises from the previous labs.

1 Toy DNSSEC

In this question we discuss a very simplified model of DNSSEC, a system for securing DNS data. The DNS system can be viewed as a database mapping each domain name to some data (next denoted the domain data, e.g. IP addresses). Domain names form a tree, e.g. the small set of domains `iana.org.`, `uclouvain.be.`, `dns.be.` (note the trailing dots, representing the root) can be represented as



The information of the DNS may be stored by a different server for each node in the tree, known as the name server for the corresponding domain. The information available on the name server is the domain data as well as the list sub-domains and their name servers.

DNSSEC associates to each domain a signing key (the domain signing key, whose corresponding verification key is known as the domain public key), which is used to sign

- the domain data (e.g., there is a signature by $sk_{\text{uclouvain.be.}}$ of `.data:A=130.104.254.195`),
- for each sub-domain: the sub-domain name concatenated to the public key of the sub-domain (e.g., there is a signature by $sk_{\text{be.}}$ of `(uclouvain.be.,  $pk_{\text{uclouvain.be.}}$ )`).

These signatures by a domain signing key can be retrieved from the name server of that domain.

1. What security property does DNSSEC guarantee? Which attack(s) does this mechanism prevent?
2. How should a DNS client verify the security of the DNS data for a domain (e.g. `dns.be`) ? What does it need to know to do so and how does it retrieve that information?
3. The DNSSEC system described above is static. However, we want to support changes in the DNS data: domains may be added and removed, domain data can be modified. How can we support this?
4. (*To do after exercise 3.*) Let us imagine that it is infeasible to use signatures and we want to design an alternative system to DNSSEC using only hash functions.
 - a) Which datastructure could we use?
 - b) How would the system work overall?
 - c) Would there be significant security and operational differences compared to DNSSEC?

2 TLS 1.3 and replay

Under which exact conditions can TLS 1.3 be vulnerable to replay attacks? Explain why the protocol designers accepted this trade-off. Propose two mitigation strategies.

3 Merkle tree

Read the wikipedia page on Merkle trees: https://en.wikipedia.org/wiki/Merkle_tree.

We work with lists of integers (e.g., $X = [9, 3, 7, 5, 2]$) and use binary Merkle trees (i.e., a Merkle tree that is also a binary tree) as a datastructure to store such lists. We denote by $X ++ Y$ the concatenation of the lists X and Y .

1. Describe the construction of the Merkle trees, taking X as an example, using a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ for some n (e.g., $n = 256$).
2. How can you use the tree to get a fixed-size identifier for any list? Explain how the identifier is computed for X . What are the security properties of this identifier? Which property should H satisfy?
3. How can you make a short proof that an element belongs to a tree? What is the size of this proof and how to verify it?
4. Let us consider the append operation: given the tree for a list L , we want to compute the tree for the list $L' = L ++ [\alpha]$, for some integer α . Explain the general process for computing the Merkle tree for L' (illustrate for $L = X$). What is the complexity of this operation? Further, how can you prove that L is a prefix

of L' (i.e., that there exists some α such that $L' = L + +[\alpha]$? What is the size of such a proof and how to verify it?

3.1)

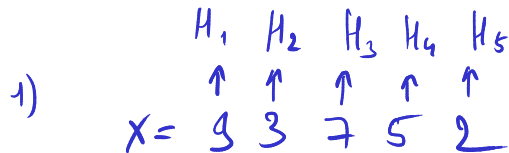
$x = 9 \ 3 \ 7 \ 5 \ 2$

3.2)

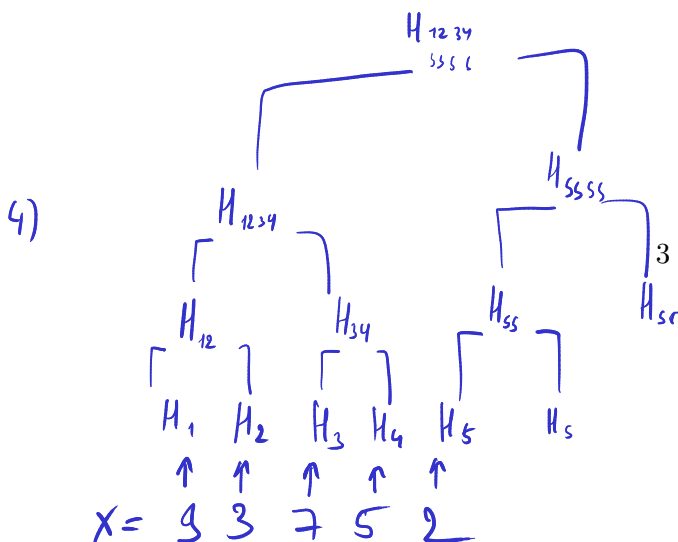
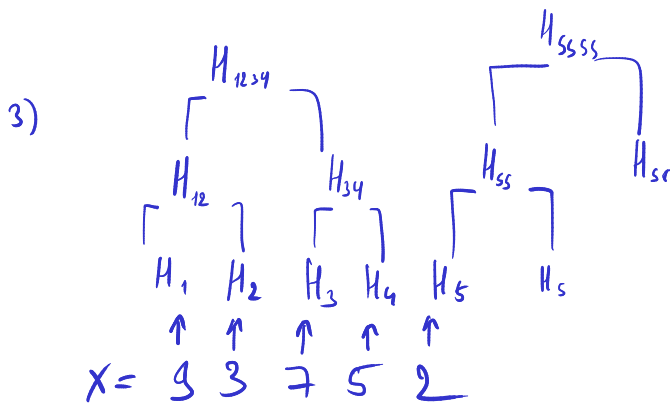
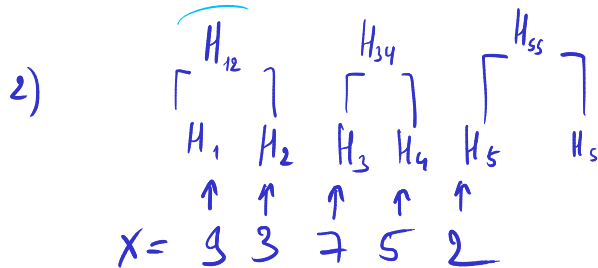
the hash of the root will be the identifier of the list, it's basically the hash with all the element of the list

Integrity : any modification to the leaf will change the root, allow to detect if list tampered
one branch of the hash tree can be downloaded at a time and the integrity of each branch can be checked immediately, even though the whole tree is not available yet

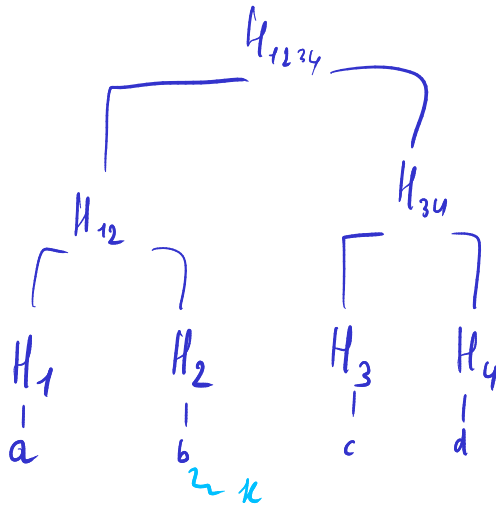
Must have a good hash function with no collision (two diff list must have two diff root hash)



$$H_{12} = \text{hash}(H_1 \parallel H_2)$$



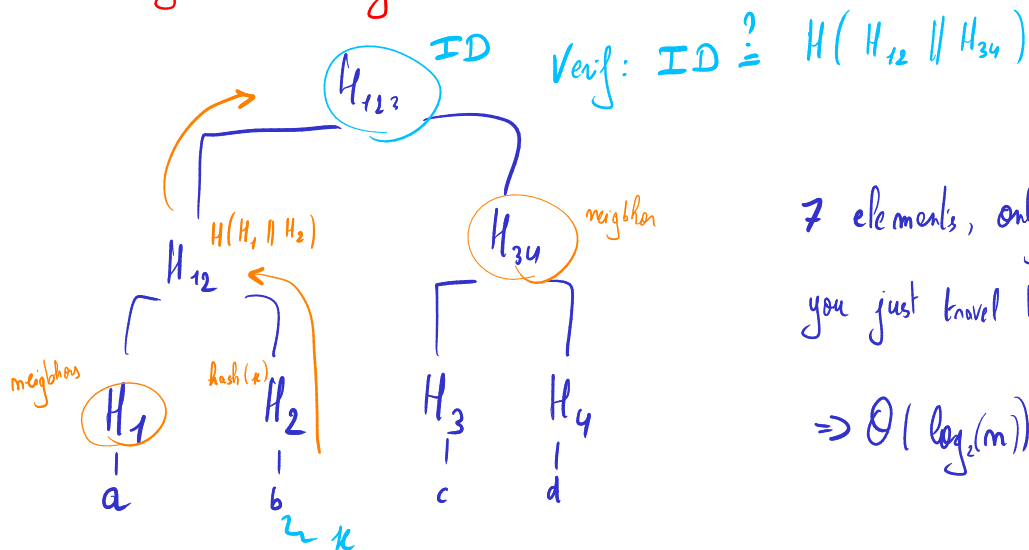
3.3



$x \in \text{Tree?}$

You must know:

- the hash of the root (Tree ID) /
- hash of neighbors as you move up /

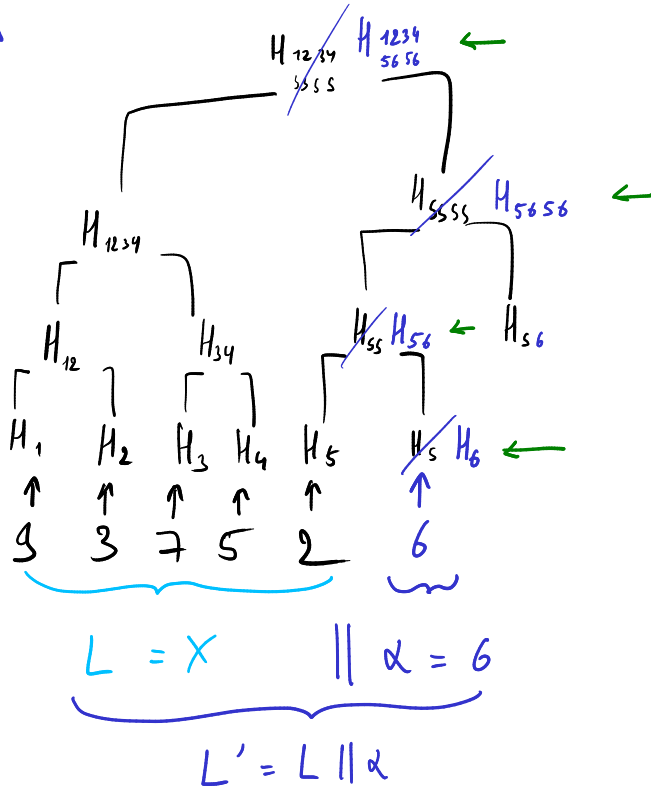


7 elements, only 3 hash computed
you just know the tree height

$$\Rightarrow O(\log_2(m))$$

3.4

Adding element

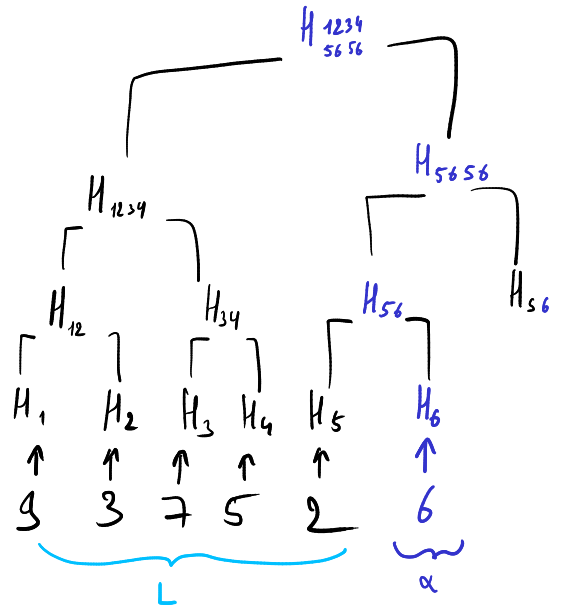
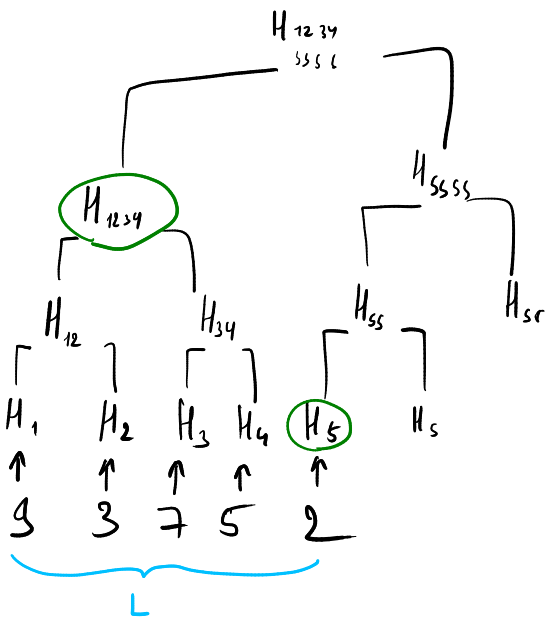


4 hashes to recompute!
 height of the tree

Old Tree

Proving L is a prefix

New Tree



We give a few nodes that allows the client to recompute the old Root hash + the new element

1) prove that those hashes generate the hold tree :

2) proves the new tree root is build on top of old tree by reusing the hashes that generate the old tree + new data

